CDynamics 0.0.1 Documentation

1 Preliminaries

Throughout our discussion $f : \hat{\mathbb{C}} \circlearrowleft$ is a non-constant holomorphic self-map of the Riemann sphere. The motivation for the following definitions and proofs of the given remarks can be found in "Dynamics of One Complex Variable - John Milnor (2006)"

Remark 1.1. $f : \hat{\mathbb{C}} \circlearrowleft$ is holomorphic $\iff$ f is rational (i.e $f = p/q$) with a finite degree ($d = \max(\deg(p), \deg(q))$).

Definition 1.1. (Julia Set): Let $f^{\circ n}$ be f's n-fold iterate. The domain of normality for the collection of iterates, $(f^{\circ k})_{k\geq 1}$, $F(f)$ is called the Fatou set of f. The complement of the Fatou set, $J(f) = \hat{\mathbb{C}} \backslash F(f)$, is called the Julia set of f.

Definition 1.2. (Periodic Point): If the orbit for some $z_0 \in \hat{\mathbb{C}}$ under f is such that $f : z_0 \to z_1 \to \cdots z_{m-1} \to z_m = z_0$, then z_0 is said to be a periodic point of period $m \geq 1$ with multiplier $\lambda = (f^{\circ m})'(z_0)$ under f.

Definition 1.3. (Classification Of Orbits): The orbit of a periodic point is said to be attracting if $|\lambda| < 1$, neutral if $|\lambda| = 1$ and repelling if $|\lambda| > 1$.

Definition 1.4. (Basin Of Attraction): If $\mathcal{O}$ is an attracting periodic orbit of period m, then the corresponding basin of attraction is the set of all points for which the successive iterates converge towards some point of $\mathcal{O}$, i.e $\text{Bas}(\mathcal{O}) \subset S = \{z \in S : \exists i = 1, \cdots m : (f^{\circ k}(z))_{k\geq 1} \to z_i \in \mathcal{O}\}$.

Definition 1.5. (Filled Julia Set): Let $\{\hat{p_i}\}_{i=1,2,3,\cdots,k\leq d+1}$ be the fixed points of f. The set of all points on the Riemann sphere that have a bounded orbit, $K(f) = \{z \in \hat{\mathbb{C}} : \mathcal{O}(z) \to p_i\}$ is called the filled Julia set of f.

Remark 1.2. The Julia set of f, is the boundary of the filled-in Julia set of f, i.e, $\partial K(f) = J(f)$

Remark 1.3. $f = p/q : \hat{\mathbb{C}} \circlearrowleft$ with algebraic degree d has atmost $d + 1$ fixed points.

2 An Algorithm to Plot Approximate Filled Julia Sets of Rational Functions

2.1 Goal:

Let $\{\hat{p_i}\}_{i=1,2,3,\cdots,k\leq d+1}$ be the fixed points a rational map $f = p/q : \hat{\mathbb{C}} \circlearrowleft$ with algebraic degree d. Our goal is to color each pixel z in a user-dictated discrete grid $[a, b] \times [c, d]$ such that the resultant image represents the filled-Julia set of f.

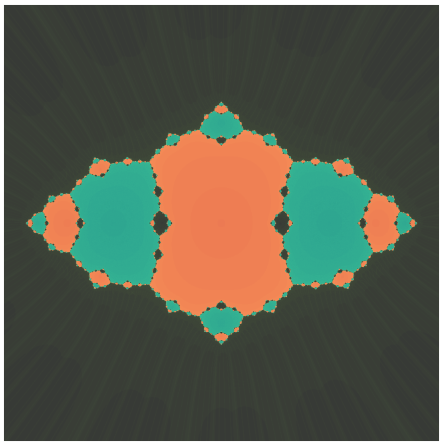
2.2 Inputs:

1. `p,q` – list, coefficients of p and q in order of increasing degree.
2. `a,b,c,d` – floats, end-point values of $[a, b] \times [c, d]$
3. `resx, resy` – int, number of points to be included in $[a, b]$ and $[c, d]$
4. `depth` – int, number of iterates to be computed per orbit.
5. `bail_out` – complex128, maximum allowed value in the orbit.
6. `cols` – list, list of RGB tuples
7. `glow` – float, determines the magnitude by which the picked RGB tuple's ligtness value is changed.

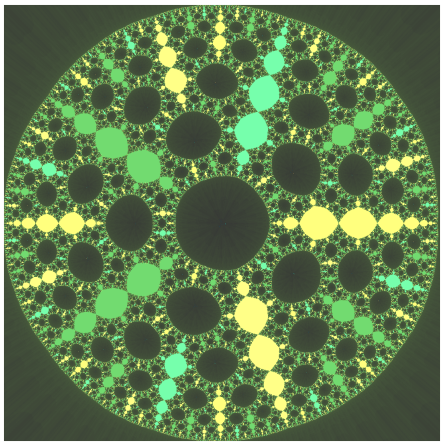
2.3 Filled Julia Sets Algorithm

Using NumPy's polynomial module, we identify and store the fixed points of $f = p/q$ (by finding the roots of $p(z) - zq(z)$) in a list `fp`. Next, for each point `i` in a discrete grid $[a, b] \times [c, d]$, we recursively compute it's image under f while the count of iterations hasn't exceeded `depth`, the orbit hasn't arrived at any value from `fp` and the penultimate iterate hasn't exceeded `bail_out`. We then store the number of iterations taken in our orbit and the closest fixed point to the last iterate in two seperate lists `convergents` and `orbit_length`. In a zero-initialized list, `colors`, each `colors[i]` is replaced by an RGB tuple in `cols` based on the value of `convergents[i]`. Further, the RGB tuple is lightened (or darkened) depending on the value of `orbit_length[i]*glow`. Lastly `colors` is reshaped into a matrix of size `resy` $\times$ `resx` and plotted using the `imshow()` function in MatPlotLib.

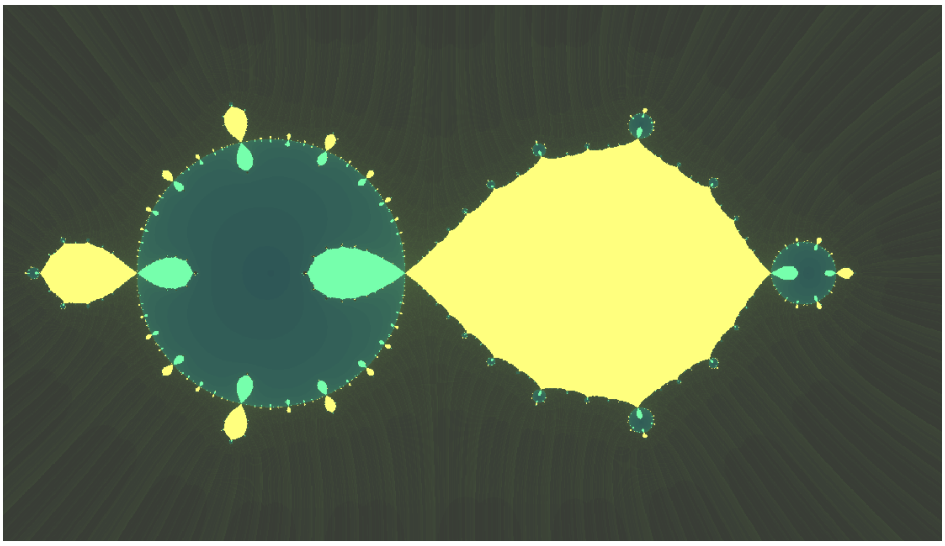
2.4 Examples



(a) $(4z^4 - 4z^6 + z^8)/(1 - 4z^2 + 2z^4)$



(b) $(z^5 + 0.01)/(z^3)$

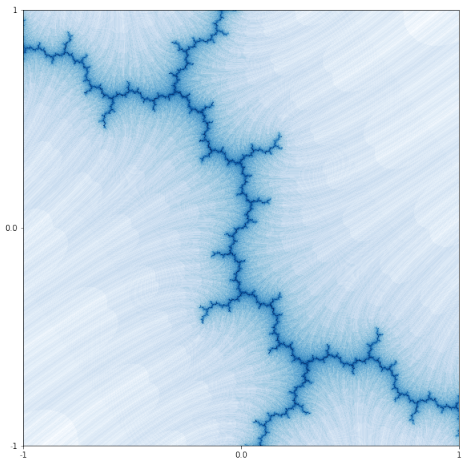


(c) $(4z^3 - z^4)/(4z - 1)$

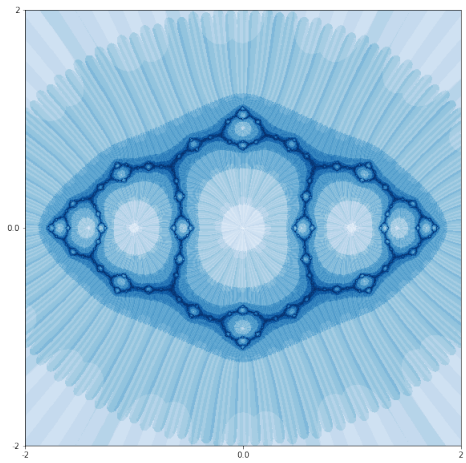
3 Other Functions In CDynamics 0.0.1

3.1 Julia Sets Algorithm

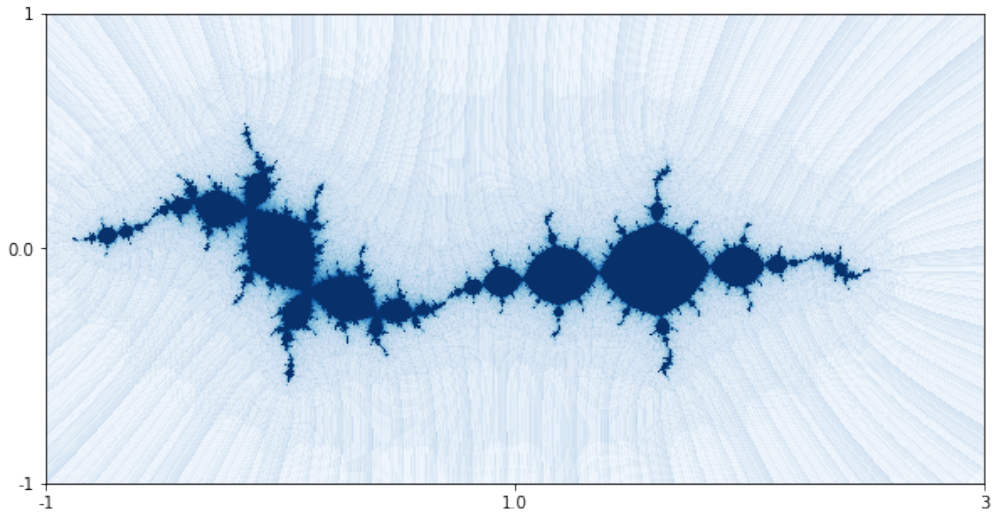
In view of Remark 1.3 it is clear that to plot the approximate Julia set of a rational function, the same algorithm that we used in the previous section will work. Except that, the coloring needs to be indifferent to which attracting point (assuming it's existence) an orbit is tending to. We implement this by reshaping `orbit_length` to `resy` $\times$ `resx` and passing it through `imshow()` along with a binary color map.



(a) $z^2 + i$



(b) $1/(z^2 - 1)$



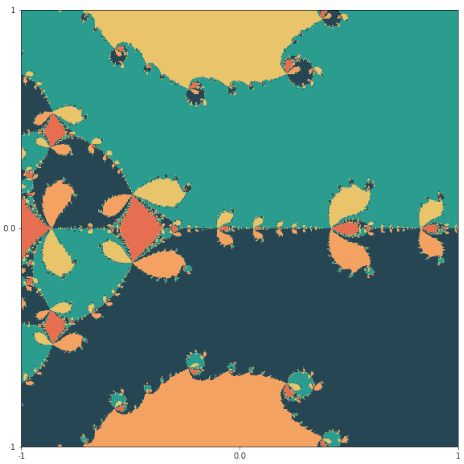
(c) $-z^3 + (2.41154 - 0.13369i)z^2 + (-0.09070 + 0.27145i)$

3.2 Examples

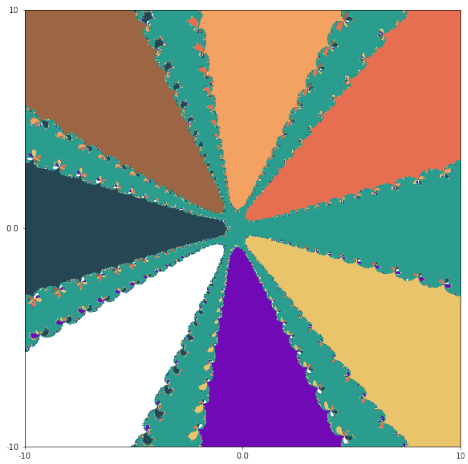
3.3 Newton's Fractal Algorithm

The same algorithm that we used in the section 2.3. will work here as well. Except that, the orbit of a point in the plane will be governed by the iteration scheme of the Newton-Raphson method.

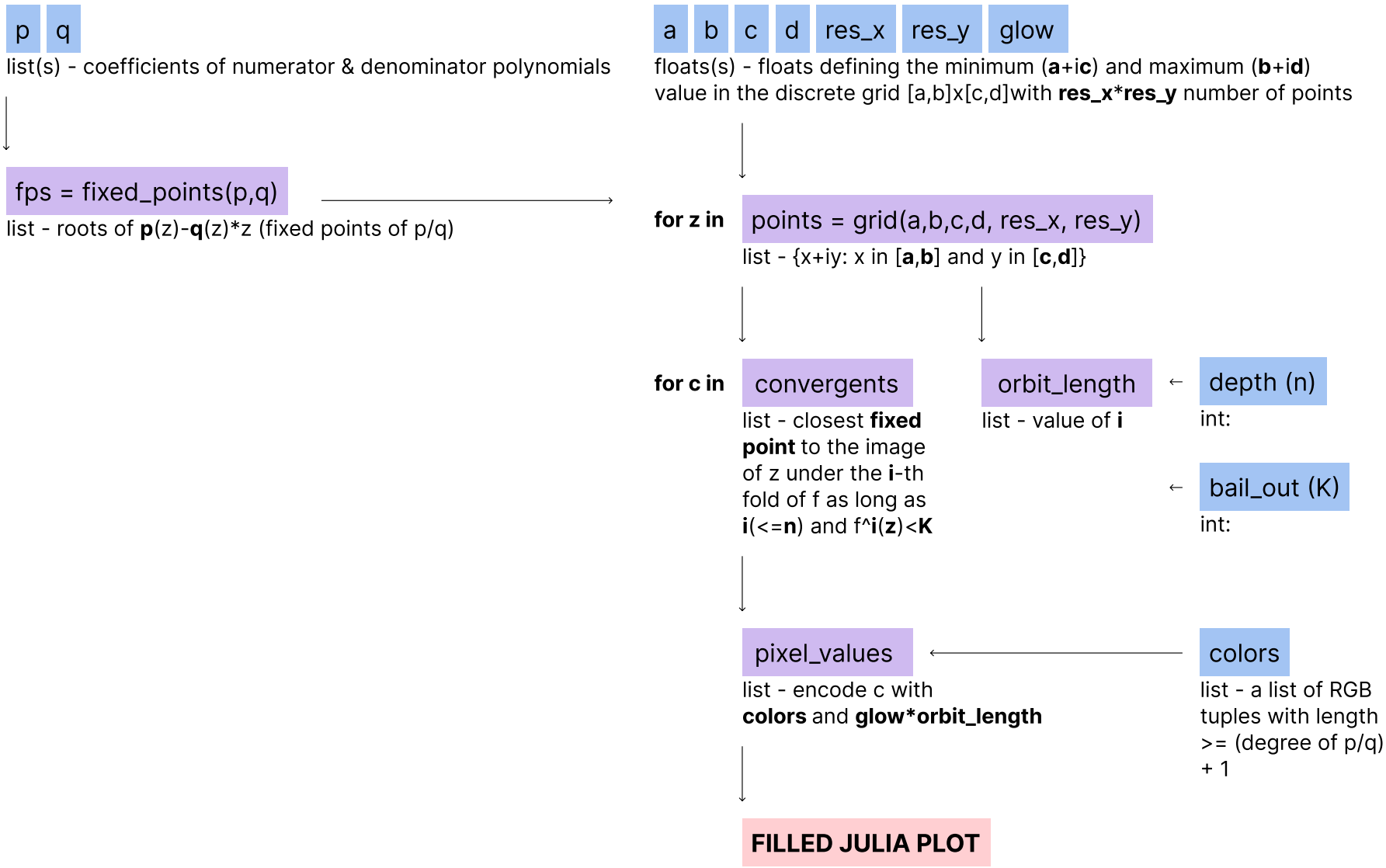
3.4 Examples



(a) $z^5 + z^2 - z + 1$



(b) $z^8 + 3z^7 + 5z^6 + 5z^5 + 4z^4 + 2^3 + z^2 + z$


p
q
list(s) - coefficients of numerator & denominator polynomials
fps = fixed_points(p,q)
list - roots of **p**(z)-**q**(z)*z (fixed points of p/q)
a
b
c
d
res_x
res_y
glow
floats(s) - floats defining the minimum (**a**+i**c**) and maximum (**b**+i**d**) value in the discrete grid [a,b]x[c,d]with **res_x*res_y** number of points
for z in
points = grid(a,b,c,d, res_x, res_y)
list - {x+iy: x in [**a**,**b**] and y in [**c**,**d**]}
for c in
convergents
list - closest **fixed point** to the image of z under the **i**-th fold of f as long as **i**(<=**n**) and f^**i**(**z**)<**K**
orbit_length
list - value of **i**
depth (n)
int:
bail_out (K)
int:
pixel_values
list - encode c with **colors** and **glow*orbit_length**
colors
list - a list of RGB tuples with length >= (degree of p/q) + 1
FILLED JULIA PLOT